



UNIVERSITY OF FRONTIER TECHNOLOGY, BANGLADESH

Department of Cyber Security Engineering

Lab Report

Name of The Experiment: Implementation of CRUD Operations using Laravel Framework

Course Code : PROG 212

Course Title : Android & Web Application Development Sessional

Submitted by,

Ragib Shahriar Abeg

ID: 2304017

Level -2, Term – 1

Session: 2023-24

Submitted to,

Md. Abdullah,

Lecturer,

Department of Cyber Security
Engineering

Experiment Name

Implementation of CRUD Operations using Laravel Framework

Objective

The objective of this experiment is to learn and implement the four fundamental database operations — Create, Read, Update, and Delete (CRUD) — using the Laravel PHP framework. Laravel is a modern, open-source web application framework that follows the MVC (Model-View-Controller) architectural pattern.

Another objective is to understand how Laravel organizes a web application through its routing system, Eloquent ORM, Blade templating engine, and Artisan command-line tool by building a fully functional student management system from scratch.

Theory

Laravel is a free, open-source PHP web framework designed for building modern web applications. It provides an expressive and elegant syntax that simplifies common web development tasks such as routing, authentication, database interaction, and session management.

Laravel follows the MVC (Model-View-Controller) design pattern which separates the application logic into three interconnected components:

- Model – Represents the data and business logic. It interacts directly with the database using Laravel's Eloquent ORM (Object Relational Mapper).
- View – Represents the user interface. Laravel uses the Blade templating engine to render HTML pages dynamically.
- Controller – Acts as the bridge between the Model and View. It handles incoming HTTP requests, processes data, and returns appropriate responses.

CRUD stands for Create, Read, Update, and Delete — the four basic operations performed on any database. In a web application, these operations correspond to specific HTTP methods:

- Create → POST request → Saves new data to the database
- Read → GET request → Retrieves and displays data from the database
- Update → PUT/PATCH request → Modifies existing data in the database
- Delete → DELETE request → Removes data from the database

Laravel's `Route::resource()` method provides a convenient way to define all CRUD routes in a single line, automatically mapping HTTP verbs to the corresponding controller methods.

List of Topics for this Experiment

- Laravel Installation and Project Setup – Creating a new Laravel project using Composer.
- Database Configuration – Connecting Laravel to a MySQL database via the `.env` file.
- Migration – Creating and running database migrations to define table structure.
- Model – Creating an Eloquent Model to interact with the students table.
- Controller – Building a `StudentController` with all CRUD methods.
- Routes – Registering resource routes in `web.php` to handle all HTTP requests.
- Blade Views – Creating index, create, and edit views using the Blade templating engine.
- CRUD Testing – Verifying all four operations work correctly in the browser.

Program / Implementation

Step 1: Project Setup

A new Laravel project named `Crudapp` was created using Composer. The development server was started using the `php artisan serve` command.

```
composer create-project laravel/laravel Crudapp
cd Crudapp
php artisan serve
```

```
D:\Project\3rd Sem\Android & Web Development\Laravel project>composer create-project laravel/laravel Crudapp
Creating a "laravel/laravel" project at ".\Crudapp"
Cannot use laravel/laravel's latest version v13.8.0 as it requires php ^8.3 which is not satisfied by your platform.
Installing laravel/laravel (v12.12.2)
- Downloading laravel/laravel (v12.12.2)
- Installing laravel/laravel (v12.12.2): Extracting archive
```

```
81 packages you are using are looking for funding.
Use the 'composer fund' command to find out more!
> @php artisan vendor:publish --tag=laravel-assets --ansi --force

[INFO] No publishable resources for tag [laravel-assets].

No security vulnerability advisories found.
> @php artisan key:generate --ansi

[INFO] Application key set successfully.

> @php -r "file_exists('database/database.sqlite') || touch('database/database.sqlite');"
> @php artisan migrate --graceful --ansi

[INFO] Preparing database.

Creating migration table ..... 9.66ms DONE

[INFO] Running migrations.

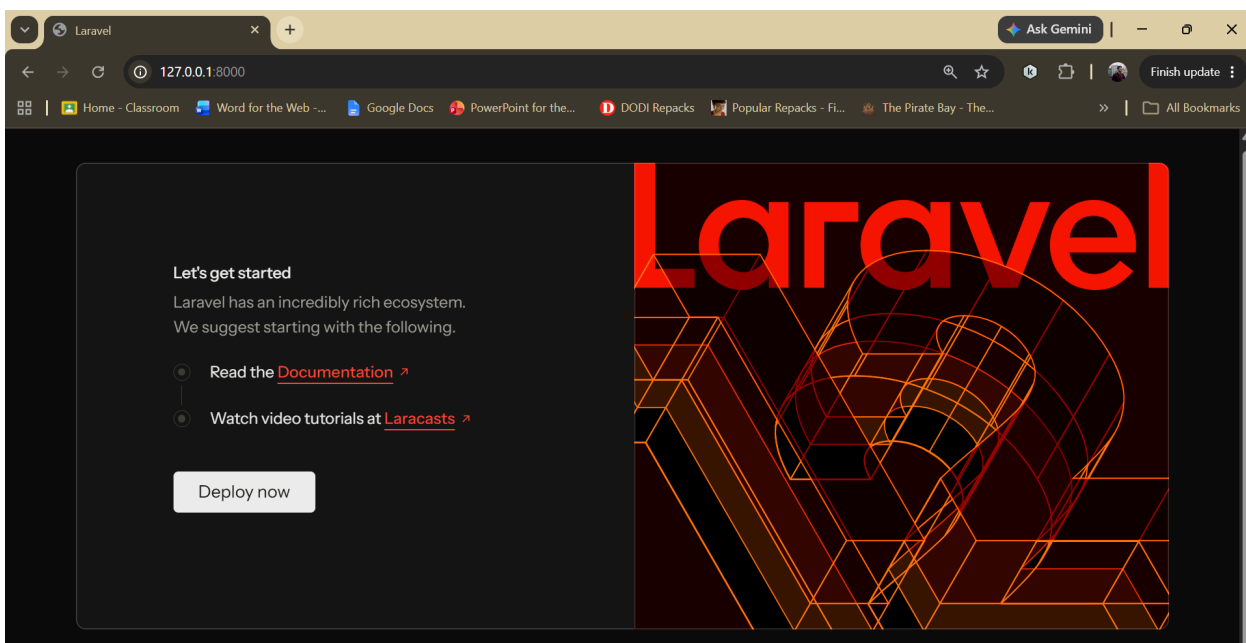
0001_01_01_000000_create_users_table ..... 22.95ms DONE
0001_01_01_000001_create_cache_table ..... 14.12ms DONE
0001_01_01_000002_create_jobs_table ..... 19.44ms DONE
```

```
PS D:\Project\3rd Sem\Android & Web Development\Laravel project\
Crudapp> php artisan serve
```

```
INFO Server running on [http://127.0.0.1:8000].
```

```
Press Ctrl+C to stop the server
```

```
2026-06-09 23:08:27 / ..... ~ 512.28ms
2026-06-09 23:08:27 /favicon.ico ..... ~ 510.46ms
```



Step 2: Database Configuration

A MySQL database named Crudapp was created using phpMyAdmin. The .env file was updated with the following database credentials to establish the connection:

```
DB_CONNECTION=mysql
DB_HOST=127.0.0.1
DB_PORT=3306
DB_DATABASE=Crudapp
DB_USERNAME=root
DB_PASSWORD=
```

XAMPP Control Panel v3.3.0 [Compiled: Apr 6th 2021]

XAMPP Control Panel v3.3.0

Service	Module	PID(s)	Port(s)	Actions
☐	Apache	32776 37104	80, 443	Stop Admin Config Logs
☐	MySQL	30908	3306	Stop Admin Config Logs
☐	FileZilla			Start Admin Config Logs
☐	Mercury			Start Admin Config Logs
☐	Tomcat			Start Admin Config Logs

12:37:12 PM [mysql] Status change detected: running
12:37:14 PM [mysql] Status change detected: stopped
12:37:14 PM [mysql] Error: MySQL shutdown unexpectedly.
12:37:14 PM [mysql] This may be due to a blocked port, missing dependencies,
12:37:14 PM [mysql] improper privileges, a crash, or a shutdown by another method.
12:37:14 PM [mysql] Press the Logs button to view error logs and check
12:37:14 PM [mysql] the Windows Event Viewer for more clues
12:37:14 PM [mysql] If you need more help, copy and post this
12:37:14 PM [mysql] entire log window on the forums
12:41:36 PM [mysql] Attempting to start MySQL app...
12:41:36 PM [mysql] Status change detected: running

phpMyAdmin Server: 127.0.0.1

Databases SQL Status User accounts Export Imp

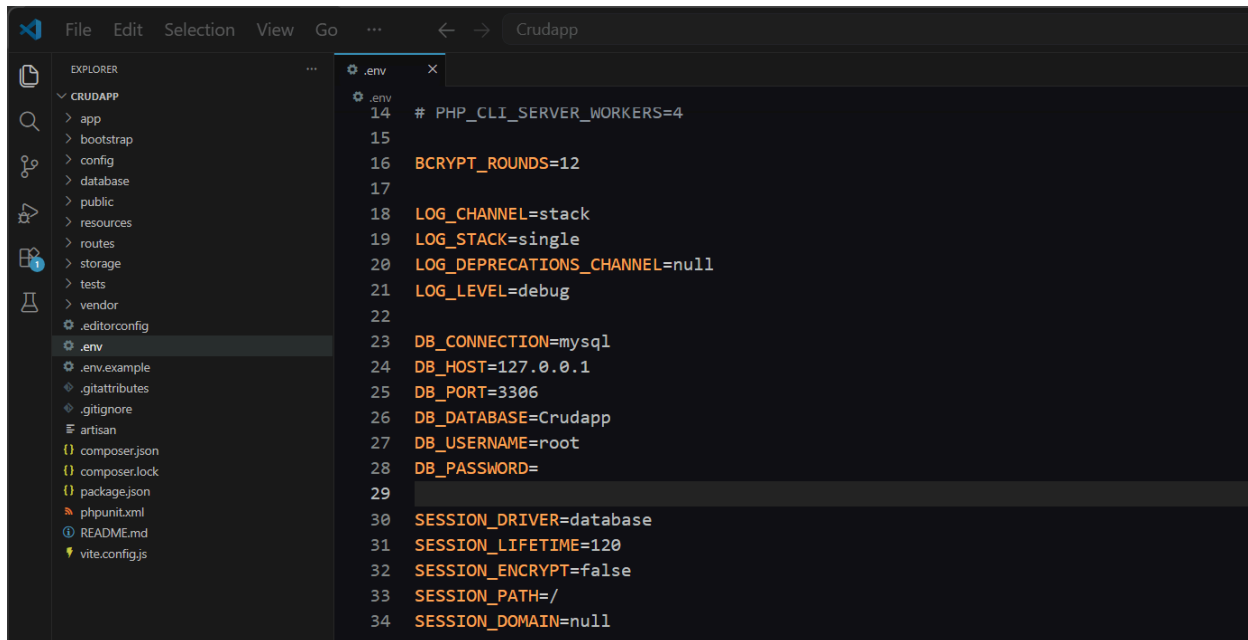
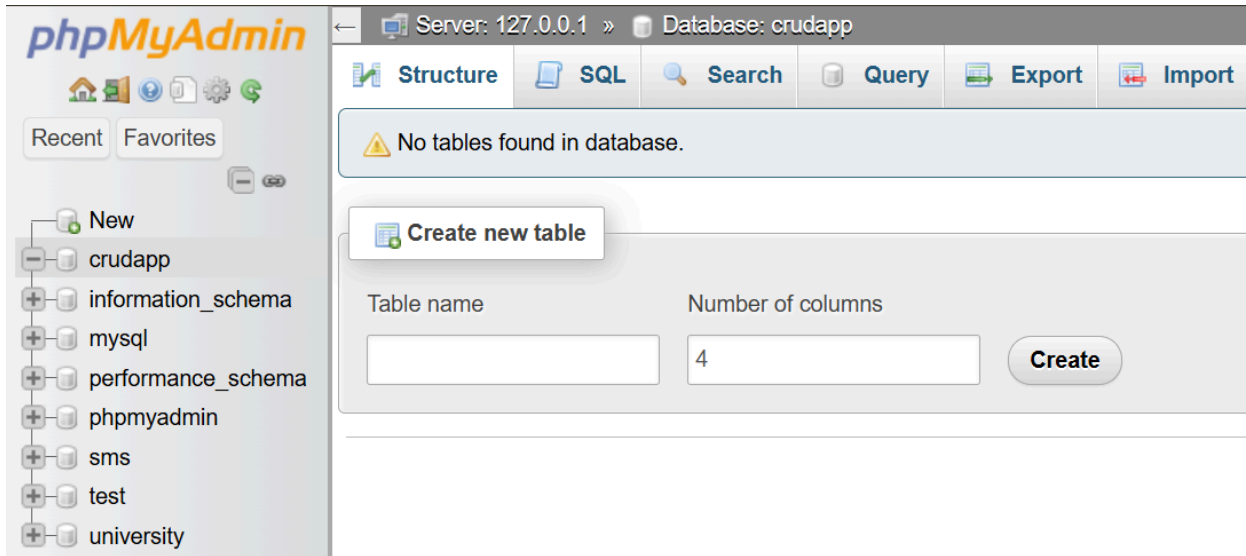
Databases

Create database

Crudapp utf8mb4_general_ci Create

☐ Check all Drop

Database	Collation	Action
☐ information_schema	utf8_general_ci	Check privileges
☐ mysql	utf8mb4_general_ci	Check privileges

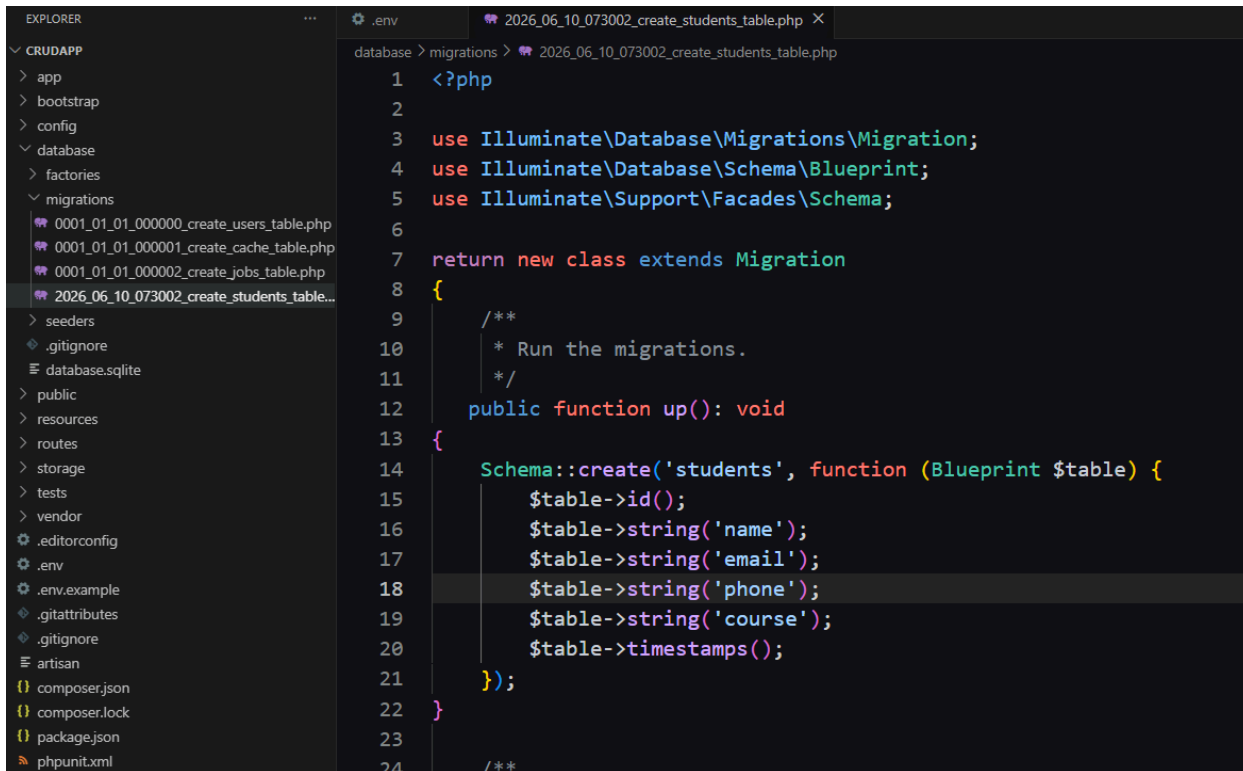


Step 3: Migration

A migration file was created to define the structure of the students table. The migration was then executed to create the table in the database.

```
php artisan make:migration create_students_table
```

```
D:\Project\3rd Sem\Android & Web Development\Laravel project\Crudapp>php artisan make:migration create_students_table
[INFO] Migration [D:\Project\3rd Sem\Android & Web Development\Laravel project\Crudapp\database\Migrations\2026_06_10_073002_create_students_table.php] created successfully.
```



```
1 <?php
2
3 use Illuminate\Database\Migrations\Migration;
4 use Illuminate\Database\Schema\Blueprint;
5 use Illuminate\Support\Facades\Schema;
6
7 return new class extends Migration
8 {
9     /**
10      * Run the migrations.
11      */
12     public function up(): void
13     {
14         Schema::create('students', function (Blueprint $table) {
15             $table->id();
16             $table->string('name');
17             $table->string('email');
18             $table->string('phone');
19             $table->string('course');
20             $table->timestamps();
21         });
22     }
23
24     /**
```

The migration file was edited to define the following columns: id, name, email, phone, course, and timestamps. The migration was then run:

```
php artisan migrate
```

```
D:\Project\3rd Sem\Android & Web Development\Laravel project\Crudapp>php artisan migrate
[INFO] Running migrations.
2026_06_10_073002_create_students_table ..... 16.06ms DONE
```

Table	Action	Rows	Type	Collation	Size	Overhead
☐ cache	★ Browse Structure Search Insert Empty Drop	0	InnoDB	utf8mb4_unicode_ci	32.0 KiB	-
☐ cache_locks	★ Browse Structure Search Insert Empty Drop	0	InnoDB	utf8mb4_unicode_ci	32.0 KiB	-
☐ failed_jobs	★ Browse Structure Search Insert Empty Drop	0	InnoDB	utf8mb4_unicode_ci	32.0 KiB	-
☐ jobs	★ Browse Structure Search Insert Empty Drop	0	InnoDB	utf8mb4_unicode_ci	32.0 KiB	-
☐ job_batches	★ Browse Structure Search Insert Empty Drop	0	InnoDB	utf8mb4_unicode_ci	16.0 KiB	-
☐ migrations	★ Browse Structure Search Insert Empty Drop	4	InnoDB	utf8mb4_unicode_ci	16.0 KiB	-
☐ password_reset_tokens	★ Browse Structure Search Insert Empty Drop	0	InnoDB	utf8mb4_unicode_ci	16.0 KiB	-
☐ sessions	★ Browse Structure Search Insert Empty Drop	0	InnoDB	utf8mb4_unicode_ci	48.0 KiB	-
☐ students	★ Browse Structure Search Insert Empty Drop	0	InnoDB	utf8mb4_unicode_ci	16.0 KiB	-
☐ users	★ Browse Structure Search Insert Empty Drop	0	InnoDB	utf8mb4_unicode_ci	32.0 KiB	-
10 tables	Sum	4	InnoDB	utf8mb4_general_ci	272.0 KiB	0 B

Step 4: Model

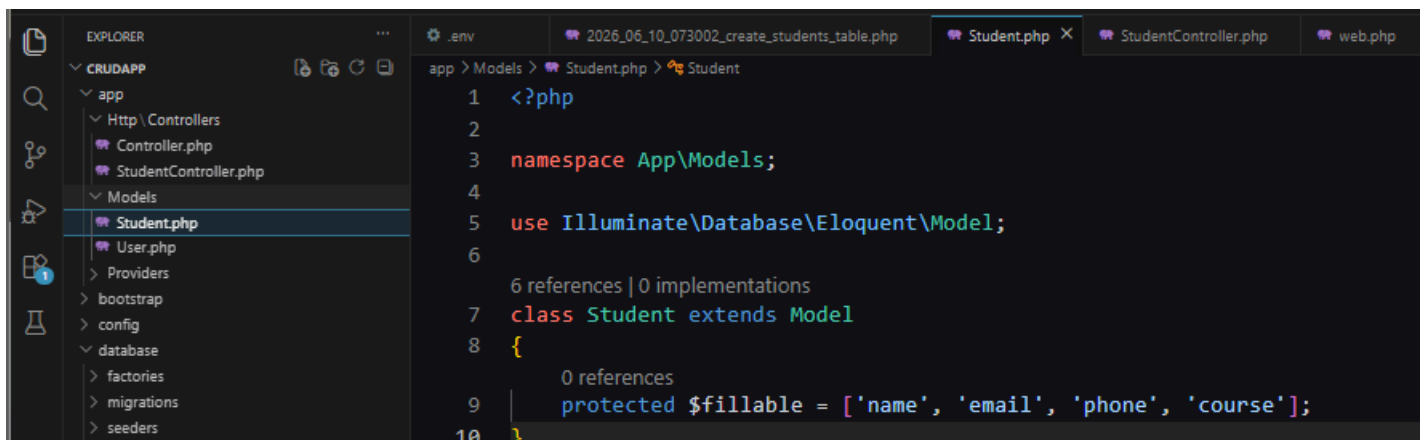
A Student Model was created using Artisan. The \$fillable property was defined to specify which columns can be mass-assigned by user input, which is required by Laravel's mass assignment protection.

```
php artisan make:model Student
```

```
D:\Project\3rd Sem\Android & Web Development\Laravel project\Crudapp>php artisan make:model Student
INFO Model [D:\Project\3rd Sem\Android & Web Development\Laravel project\Crudapp\app\Models\Student.php] created successfully.
```

Inside Student.php:

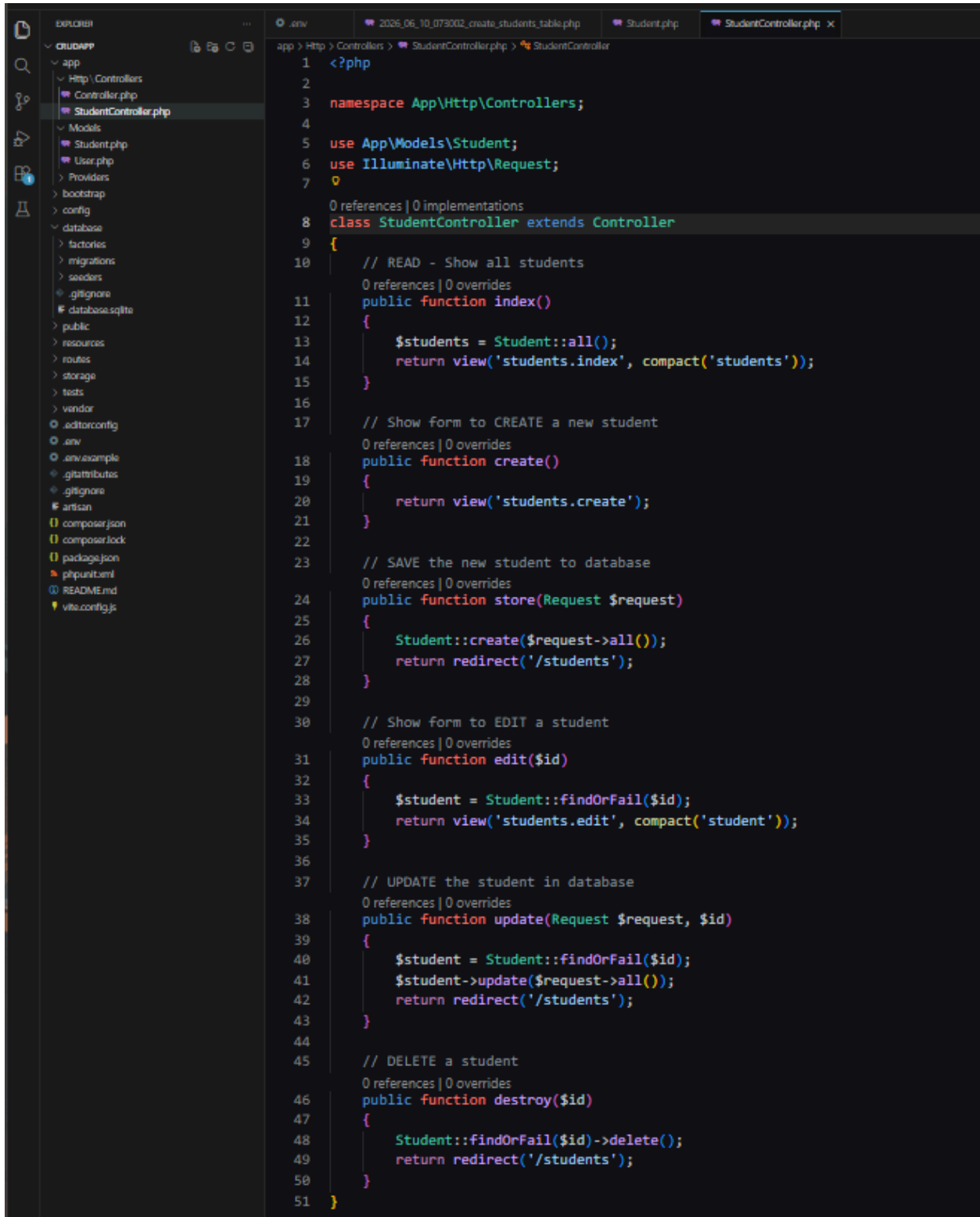
```
protected $fillable = ['name', 'email', 'phone', 'course'];
```



Step 5: Controller

A StudentController was created containing all six CRUD methods: index(), create(), store(), edit(), update(), and destroy(). Each method handles a specific operation on the students table.

```
php artisan make:controller StudentController
```

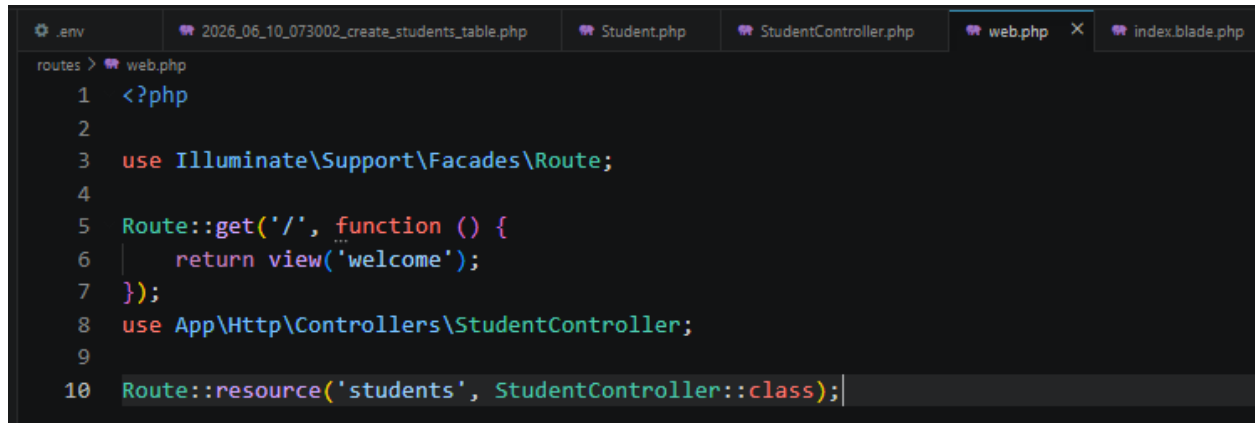


```
1 <?php
2
3 namespace App\Http\Controllers;
4
5 use App\Models\Student;
6 use Illuminate\Http\Request;
7
8 class StudentController extends Controller
9 {
10     // READ - Show all students
11     0 references | 0 overrides
12     public function index()
13     {
14         $students = Student::all();
15         return view('students.index', compact('students'));
16     }
17
18     // Show form to CREATE a new student
19     0 references | 0 overrides
20     public function create()
21     {
22         return view('students.create');
23     }
24
25     // SAVE the new student to database
26     0 references | 0 overrides
27     public function store(Request $request)
28     {
29         Student::create($request->all());
30         return redirect('/students');
31     }
32
33     // Show form to EDIT a student
34     0 references | 0 overrides
35     public function edit($id)
36     {
37         $student = Student::findOrFail($id);
38         return view('students.edit', compact('student'));
39     }
40
41     // UPDATE the student in database
42     0 references | 0 overrides
43     public function update(Request $request, $id)
44     {
45         $student = Student::findOrFail($id);
46         $student->update($request->all());
47         return redirect('/students');
48     }
49
50     // DELETE a student
51     0 references | 0 overrides
52     public function destroy($id)
53     {
54         Student::findOrFail($id)->delete();
55         return redirect('/students');
56     }
57 }
```

Step 6: Routes

A single resource route was registered in routes/web.php that automatically generates all required CRUD routes — eliminating the need to define each route individually.

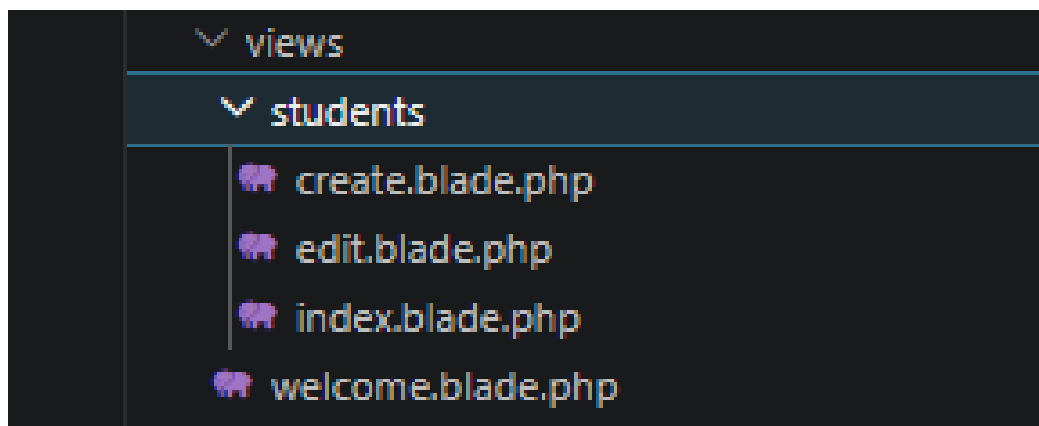
```
Route::resource('students', StudentController::class);
```



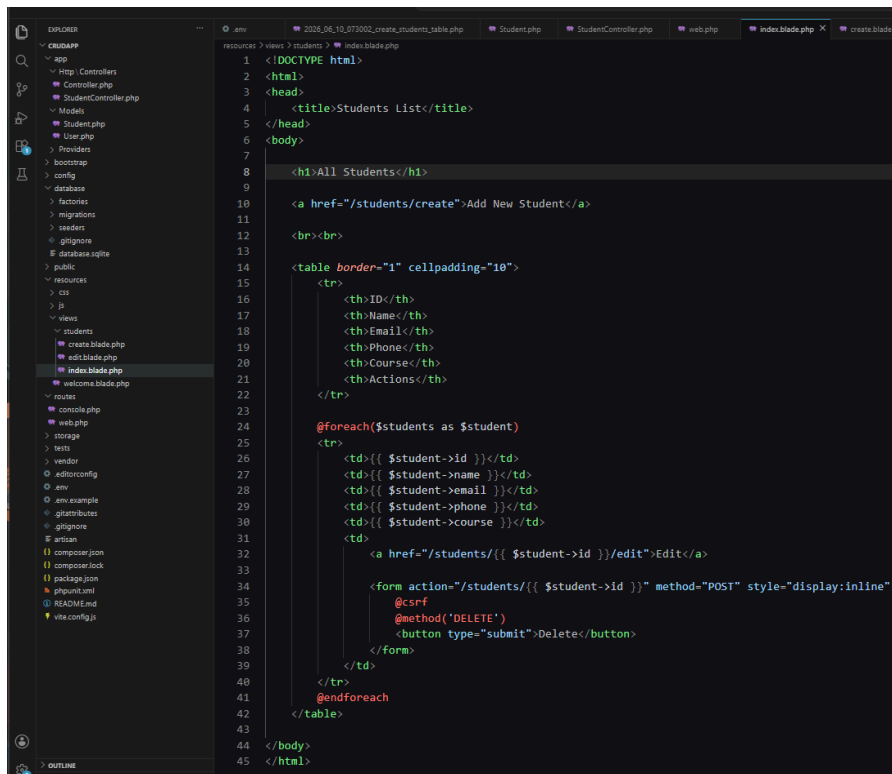
```
routes > web.php
1  <?php
2
3  use Illuminate\Support\Facades\Route;
4
5  Route::get('/', function () {
6      return view('welcome');
7  });
8  use App\Http\Controllers\StudentController;
9
10 Route::resource('students', StudentController::class);
```

Step 7: Blade Views

Three Blade view files were created inside the resources/views/students/ directory:



- index.blade.php – Displays all students in a table with Edit and Delete buttons.

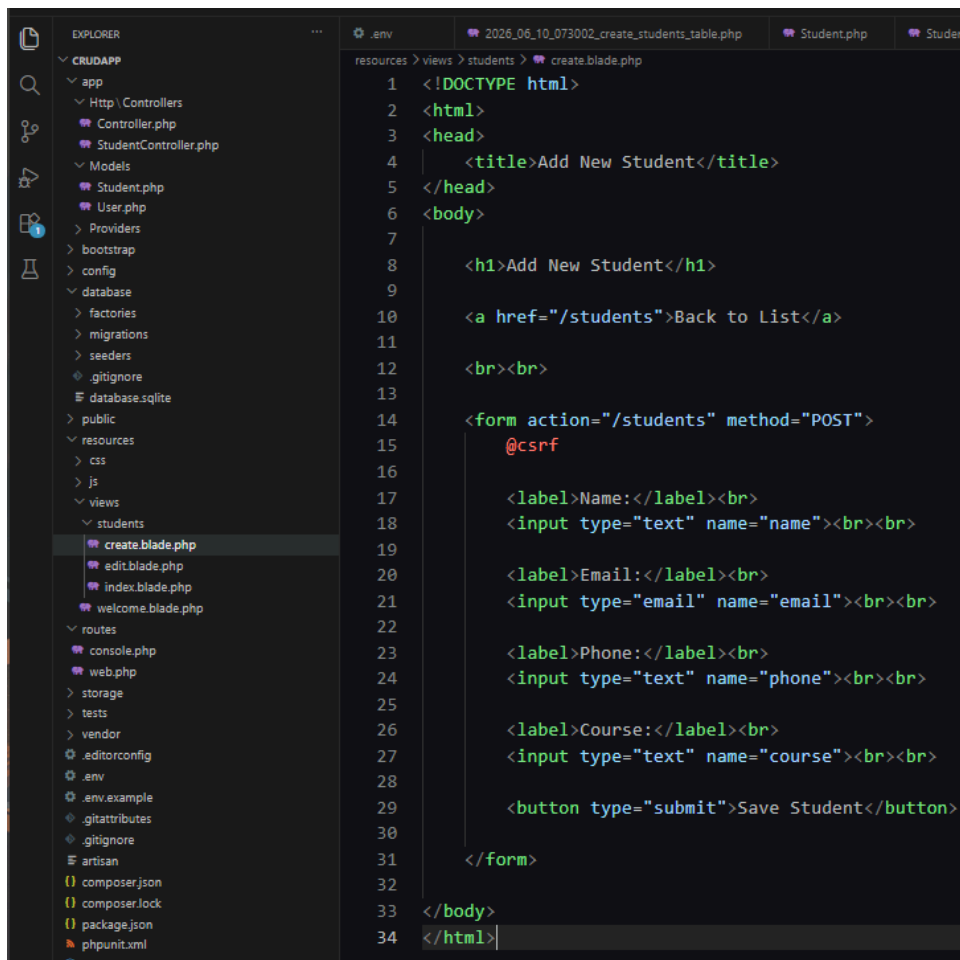


```

1 <!DOCTYPE html>
2 <html>
3 <head>
4 <title>Students List</title>
5 </head>
6 <body>
7
8 <h1>All Students</h1>
9
10 <a href="/students/create">Add New Student</a>
11
12 <br><br>
13
14 <table border="1" cellpadding="10">
15
16 <tr>
17 <th>ID</th>
18 <th>Name</th>
19 <th>Email</th>
20 <th>Phone</th>
21 <th>Course</th>
22 <th>Actions</th>
23 </tr>
24
25 @foreach($students as $student)
26 <tr>
27 <td>{{ $student->id }}</td>
28 <td>{{ $student->name }}</td>
29 <td>{{ $student->email }}</td>
30 <td>{{ $student->phone }}</td>
31 <td>{{ $student->course }}</td>
32 <td>
33 <a href="/students/{{ $student->id }}/edit">Edit</a>
34
35 <form action="/students/{{ $student->id }}" method="POST" style="display:inline">
36 @csrf
37 @method('DELETE')
38 <button type="submit">Delete</button>
39 </form>
40 </td>
41 </tr>
42 @endforeach
43 </table>
44
45 </body>
46 </html>

```

- create.blade.php – A form with Name, Email, Phone, and Course fields to add a new student.



```

1 <!DOCTYPE html>
2 <html>
3 <head>
4 <title>Add New Student</title>
5 </head>
6 <body>
7
8 <h1>Add New Student</h1>
9
10 <a href="/students">Back to List</a>
11
12 <br><br>
13
14 <form action="/students" method="POST">
15 @csrf
16
17 <label>Name:</label><br>
18 <input type="text" name="name"><br><br>
19
20 <label>Email:</label><br>
21 <input type="email" name="email"><br><br>
22
23 <label>Phone:</label><br>
24 <input type="text" name="phone"><br><br>
25
26 <label>Course:</label><br>
27 <input type="text" name="course"><br><br>
28
29 <button type="submit">Save Student</button>
30
31 </form>
32
33 </body>
34 </html>

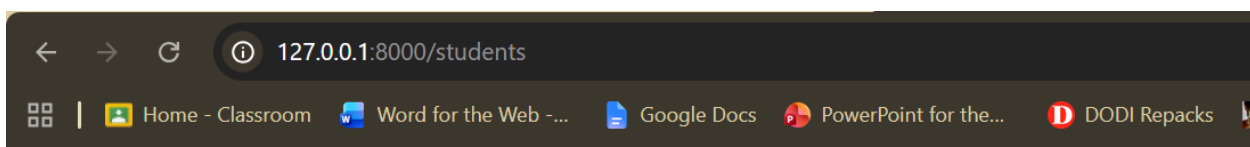
```

- edit.blade.php – A pre-filled form that allows editing of an existing student's information.

```

1 <!DOCTYPE html>
2 <html>
3 <head>
4   <title>Edit Student</title>
5 </head>
6 <body>
7
8   <h1>Edit Student</h1>
9
10  <a href="/students">Back to List</a>
11
12  <br><br>
13
14  <form action="/students/{{ $student->id }}" method="POST">
15    @csrf
16    @method('PUT')
17
18    <label>Name:</label><br>
19    <input type="text" name="name" value="{{ $student->name }}"><br><br>
20
21    <label>Email:</label><br>
22    <input type="email" name="email" value="{{ $student->email }}"><br><br>
23
24    <label>Phone:</label><br>
25    <input type="text" name="phone" value="{{ $student->phone }}"><br><br>
26
27    <label>Course:</label><br>
28    <input type="text" name="course" value="{{ $student->course }}"><br><br>
29
30    <button type="submit">Update Student</button>
31
32  </form>
33
34 </body>
35 </html>

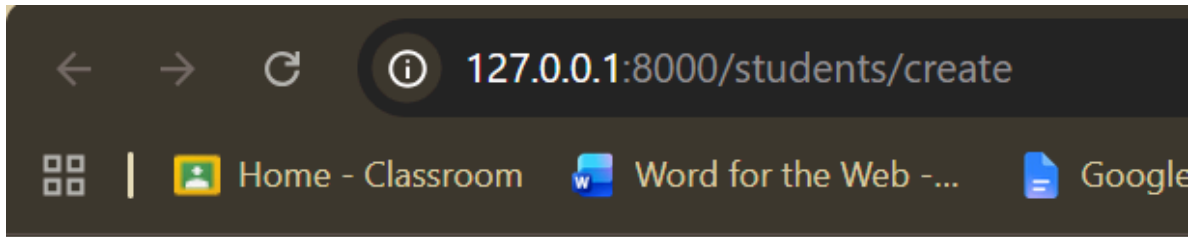
```



All Students

[Add New Student](#)

ID	Name	Email	Phone	Course	Actions
----	------	-------	-------	--------	---------



Add New Student

[Back to List](#)

Name:

Email:

Phone:

Course:

Save Student

Add New Student

[Back to List](#)

Name:

Ragib Shahriar Abeg

Email:

ragibabeg1@gmail.com

Phone:

01602086423

Course:

Web and Android Developm

Save Student

All Students

[Add New Student](#)

ID	Name	Email	Phone	Course	Actions
1	Ragib Shahriar Abeg	ragibabeg1@gmail.com	01602086423	Web and Android Development	Edit Delete

Edit Student

[Back to List](#)

Name:

Email:

Phone:

Course:

Practical Applications

- Used to build real-world data management systems such as student portals, employee management systems, and inventory systems.
- Demonstrates how web applications interact with databases to store, retrieve, modify, and remove records.
- Laravel's MVC pattern promotes clean, maintainable, and scalable code structure suitable for professional projects.
- The Blade templating engine simplifies dynamic HTML rendering, making front-end integration faster and cleaner.
- Resource routing reduces code repetition and follows RESTful API design conventions widely used in industry.
- Migrations ensure database version control, allowing teams to track and reproduce database structure changes.

Discussion

During this experiment, a complete CRUD web application was built step by step using the Laravel framework. Starting from project creation and database configuration, the application was developed through migrations, model creation, controller logic, route registration, and Blade view design.

All four CRUD operations were successfully tested in the browser. A new student record was created using the Add New Student form and saved to the MySQL database. The record was then displayed on the All Students list page. The Edit button opened a pre-filled form where the course field was modified and updated in the database. Finally, the Delete button removed the record from the database and redirected back to the list page.

The experiment demonstrated how Laravel's built-in tools such as Eloquent ORM, Artisan commands, resource routes, and Blade templates work together to simplify and speed up web application development significantly.

Conclusion

This experiment successfully demonstrated the implementation of CRUD operations using the Laravel PHP framework. By building a Student Management System from scratch, a practical understanding of Laravel's MVC architecture, database migration, Eloquent ORM, RESTful routing, and Blade templating was gained.

The knowledge acquired from this experiment forms a strong foundation for developing more complex and feature-rich web applications in future projects involving Laravel and other modern PHP framework.